

Java Security Cheat Sheet

Injection Prevention in Java

This section aims to provide tips to handle *Injection* in Java application code.

Sample code used in tips is located [here](#).

What is Injection

Injection in OWASP Top 10 is defined as following:

Consider anyone who can send untrusted data to the system, including external users, internal users, and administrators.

General advice to prevent Injection

The following point can be applied, in a general way, to prevent *Injection* issue:

1. Apply **Input Validation** (using allowlist approach) combined with **Output Sanitizing+Escaping** on user input/output.
2. If you need to interact with system, try to use API features provided by your technology stack (Java / .Net / PHP..) instead of building command.

Additional advice is provided on this [cheatsheet](#).

Specific Injection types

Examples in this section will be provided in Java technology (see Maven project associated) but advice is applicable to others technologies like .Net / PHP / Ruby / Python...

SQL

Symptom

Injection of this type occur when the application uses untrusted user input to build an SQL query using a String and execute it.

How to prevent

Use *Query Parameterization* in order to prevent injection.

Example

```

/*No DB framework used here in order to show the real use of
   Prepared Statement from Java API*/
/*Open connection with H2 database and use it*/
Class.forName("org.h2.Driver");
String jdbcUrl = "jdbc:h2:file:" + new File(".").getAbsolutePath() +
"/target/db";
try (Connection con = DriverManager.getConnection(jdbcUrl)) {

    /* Sample A: Select data using Prepared Statement*/
    String query = "select * from color where friendly_name = ?";
    List<String> colors = new ArrayList<>();
    try (PreparedStatement pStatement = con.prepareStatement(query)) {
        pStatement.setString(1, "yellow");
        try (ResultSet rSet = pStatement.executeQuery()) {
            while (rSet.next()) {
                colors.add(rSet.getString(1));
            }
        }
    }

    /* Sample B: Insert data using Prepared Statement*/
    query = "insert into color(friendly_name, red, green, blue) values(?, ?,
?, ?)";
    int insertedRecordCount;
    try (PreparedStatement pStatement = con.prepareStatement(query)) {
        pStatement.setString(1, "orange");
        pStatement.setInt(2, 239);
        pStatement.setInt(3, 125);
        pStatement.setInt(4, 11);
        insertedRecordCount = pStatement.executeUpdate();
    }

    /* Sample C: Update data using Prepared Statement*/
    query = "update color set blue = ? where friendly_name = ?";
    int updatedRecordCount;
    try (PreparedStatement pStatement = con.prepareStatement(query)) {
        pStatement.setInt(1, 10);
        pStatement.setString(2, "orange");
        updatedRecordCount = pStatement.executeUpdate();
    }

    /* Sample D: Delete data using Prepared Statement*/
    query = "delete from color where friendly_name = ?";
    int deletedRecordCount;
    try (PreparedStatement pStatement = con.prepareStatement(query)) {
        pStatement.setString(1, "orange");
        deletedRecordCount = pStatement.executeUpdate();
    }

}

```

References

- [SQL Injection Prevention Cheat Sheet](#)

JPA

Symptom

Injection of this type occur when the application uses untrusted user input to build a JPA query using a String and execute it. It's quite similar to SQL injection but here the altered language is not SQL but JPA QL.

How to prevent

Use Java Persistence Query Language **Query Parameterization** in order to prevent injection.

Example

```
EntityManager entityManager = null;
try {
    /* Get a ref on EntityManager to access DB */
    entityManager =
        Persistence.createEntityManagerFactory("testJPA").createEntityManager();

    /* Define parameterized query prototype using named parameter to enhance
    readability */
    String queryPrototype = "select c from Color c where c.friendlyName =
        :colorName";

    /* Create the query, set the named parameter and execute the query */
    Query queryObject = entityManager.createQuery(queryPrototype);
    Color c = (Color) queryObject.setParameter("colorName",
        "yellow").getSingleResult();
} finally {
    if (entityManager != null && entityManager.isOpen()) {
        entityManager.close();
    }
}
```

References

- [SQLi and JPA](#)

Operating System

Symptom

Injection of this type occur when the application uses untrusted user input to build an Operating System command using a String and execute it.

How to prevent

Use technology stack **API** in order to prevent injection.

Example

```
/* The context taken is, for example, to perform a PING against a computer.
 * The prevention is to use the feature provided by the Java API instead of
building
 * a system command as String and execute it */
InetAddress host = InetAddress.getByName("localhost");
var reachable = host.isReachable(5000);
```

References

- [Command Injection](#)

XML: XPath Injection

Symptom

Injection of this type occur when the application uses untrusted user input to build a XPath query using a String and execute it.

How to prevent

Use **XPath Variable Resolver** in order to prevent injection.

Example

Variable Resolver implementation.

```
/**
 * Resolver in order to define parameter for XPATH expression.
 *
 */
public class SimpleVariableResolver implements XPathVariableResolver {

    private final Map<QName, Object> vars = new HashMap<QName, Object>();

    /**
     * External methods to add parameter
     *
     * @param name Parameter name
     * @param value Parameter value
     */
    public void addVariable(QName name, Object value) {
        vars.put(name, value);
    }

    /**
     * {@inheritDoc}
     *
     * @see
     javax.xml.xpath.XPathVariableResolver#resolveVariable(javax.xml.namespace.QName)
    */
}
```

```

    */
    public Object resolveVariable(QName variableName) {
        return vars.get(variableName);
    }
}

```

Code using it to perform XPath query.

```

/*Create a XML document builder factory*/
DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();

/*Disable External Entity resolution for different cases*/
//Do not performed here in order to focus on variable resolver code
//but do it for production code !

/*Load XML file*/
DocumentBuilder builder = dbf.newDocumentBuilder();
Document doc = builder.parse(new File("src/test/resources/SampleXPath.xml"));

/* Create and configure parameter resolver */
String bid = "bk102";
SimpleVariableResolver variableResolver = new SimpleVariableResolver();
variableResolver.addVariable(new QName("bookId"), bid);

/*Create and configure XPATH expression*/
XPath xpath = XPathFactory.newInstance().newXPath();
xpath.setXPathVariableResolver(variableResolver);
XPathExpression xpathExpression = xpath.compile("//book[@id=$bookId]");

/* Apply expression on XML document */
Object nodes = xpathExpression.evaluate(doc, XPathConstants.NODESET);
NodeList nodeList = (NodeList) nodes;
Element book = (Element)nodeList.item(0);
var containsRalls = book.getTextContent().contains("Ralls, Kim");

```

References

- [XPath Injection](#)

HTML/JavaScript/CSS

Symptom

Injection of this type occur when the application uses untrusted user input to build an HTTP response and sent it to browser.

How to prevent

Either apply strict input validation (allowlist approach) or use output sanitizing+escaping if input validation is not possible (combine both every time is possible).

Example

```

/*
INPUT WAY: Receive data from user
Here it's recommended to use strict input validation using allowlist approach.
In fact, you ensure that only allowed characters are part of the input
received.
*/

String userInput = "You user login is owasp-user01";

/* First we check that the value contains only expected character*/
if (!Pattern.matches("[a-zA-Z0-9\\s\\-]{1,50}", userInput))
{
    return false;
}

/* If the first check pass then ensure that potential dangerous character
that we have allowed for business requirement are not used in a dangerous way.
For example here we have allowed the character '-', and, this can
be used in SQL injection so, we
ensure that this character is not used in a continuous form.
Use the API COMMONS LANG v3 to help in String analysis...
*/
If (0 != StringUtils.countMatches(userInput.replace(" ", ""), "--"))
{
    return false;
}

/*
OUTPUT WAY: Send data to user
Here we escape + sanitize any data sent to user
Use the OWASP Java HTML Sanitizer API to handle sanitizing
Use the OWASP Java Encoder API to handle HTML tag encoding (escaping)
*/

String outputToUser = "You <p>user login</p> is <strong>owasp-
user01</strong>";
outputToUser += "<script>alert(22);</script><img src='#'
onload='javascript:alert(23);'>";

/* Create a sanitizing policy that only allow tag '<p>' and '<strong>*/
PolicyFactory policy = new HtmlPolicyBuilder().allowElements("p",
"strong").toFactory();

/* Sanitize the output that will be sent to user*/
String safeOutput = policy.sanitize(outputToUser);

/* Encode HTML Tag*/
safeOutput = Encode.forHtml(safeOutput);
String finalSafeOutputExpected = "You <p>user login</p> is <strong>owasp-
user01</strong>";
if (!finalSafeOutputExpected.equals(safeOutput))
{
    return false;
}

```

References

- [XSS](#)
- [OWASP Java HTML Sanitizer](#)
- [OWASP Java Encoder](#)
- [Java RegEx](#)

LDAP

A dedicated [cheatsheet](#) has been created.

NoSQL

Symptom

Injection of this type occur when the application uses untrusted user input to build a NoSQL API call expression.

How to prevent

As there many NoSQL database system and each one use an API for call, it's important to ensure that user input received and used to build the API call expression does not contain any character that have a special meaning in the target API syntax. This in order to avoid that it will be used to escape the initial call expression in order to create another one based on crafted user input. It's also important to not use string concatenation to build API call expression but use the API to create the expression.

Example - MongoDB

```
/* Here use MongoDB as target NoSQL DB */
String userInput = "Brooklyn";

/* First ensure that the input do no contains any special characters
for the current NoSQL DB call API,
here they are: ' " \ ; { } $
*/
//Avoid regexp this time in order to made validation code
//more easy to read and understand...
ArrayList < String > specialCharsList = new ArrayList < String > () {
    {
        add("'");
        add("\");
        add("\\");
        add(";");
        add("{");
        add("}");
        add("$");
    }
};

for (String specChar: specialCharsList) {
```

```

        if (userInput.contains(specChar)) {
            return false;
        }
    }

    //Add also a check on input max size
    if (!userInput.length() <= 50)
    {
        return false;
    }

    /* Then perform query on database using API to build expression */
    //Connect to the local MongoDB instance
    try(MongoClient mongoClient = new MongoClient()){
        MongoDBDatabase db = mongoClient.getDatabase("test");
        //Use API query builder to create call expression
        //Create expression
        Bson expression = eq("borough", userInput);
        //Perform call
        FindIterable<org.bson.Document> restaurants =
        db.getCollection("restaurants").find(expression);
        //Verify result consistency
        restaurants.forEach(new Block<org.bson.Document>() {
            @Override
            public void apply(final org.bson.Document doc) {
                String restBorough = (String)doc.get("borough");
                if (!"Brooklyn".equals(restBorough))
                {
                    return false;
                }
            }
        });
    }
}

```

References

- [Testing for NoSQL injection](#)
- [SQL and NoSQL Injection](#)
- [No SQL, No Injection?](#)

Log Injection

Symptom

[Log Injection](#) occurs when an application includes untrusted data in an application log message (e.g., an attacker can cause an additional log entry that looks like it came from a completely different user, if they can inject CRLF characters in the untrusted data). More information about this attack is available on the OWASP [Log Injection](#) page.

How to prevent

To prevent an attacker from writing malicious content into the application log, apply defenses such as:

- Use structured log formats, such as JSON, instead of unstructured text formats. Unstructured formats are susceptible to **Carriage Return (CR)** and **Line Feed (LF)** injection (see [CWE-93](#)).
- Limit the size of the user input value used to create the log message.
- Make sure [all XSS defenses](#) are applied when viewing log files in a web browser.

Example using Log4j Core 2

The recommended logging policy for a production environment is sending logs to a network socket using the structured [JSON Template Layout](#) introduced in [Log4j 2.14.0](#) and limit the size of strings to 500 bytes using the [maxStringLength](#) configuration attribute:

```
<?xml version="1.0" encoding="UTF-8"?>
<Configuration xmlns="https://logging.apache.org/xml/ns"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    https://logging.apache.org/xml/ns
    https://logging.apache.org/xml/ns/log4j-config-2.xsd">
  <Appenders>
    <Socket name="SOCKET"
      host="localhost"
      port="12345">
      <!-- Limit the size of any string field in the produced JSON document to
      500 bytes -->
      <JsonTemplateLayout maxStringLength="500"
        nullEventDelimiterEnabled="true"/>
    </Socket>
  </Appenders>
  <Loggers>
    <Root level="DEBUG">
      <AppenderRef ref="SOCKET"/>
    </Root>
  </Loggers>
</Configuration>
```

See [Integration with service-oriented architectures](#) on [Log4j website](#) for more tips.

Usage of the logger at code level:

```
import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;
...
// Most common way to declare a logger
private static final LOGGER = LogManager.getLogger();
// GOOD!
//
// Use parameterized logging to add user data to a message
// The pattern should be a compile-time constant
```

```
logger.warn("Login failed for user {}", username);
// BAD!
//
// Don't mix string concatenation and parameters
// If `username` contains `{}`, the exception will leak into the message
logger.warn("Failure for user " + username + " and role {}", role, ex);
...
```

See [Log4j API Best Practices](#) for more information.

Example using Logback

The recommended logging policy for a production environment is using the structured [JsonEncoder](#) introduced in [Logback 1.3.8](#). In the example below, Logback is configured to roll on 10 log files of 5 MiB each:

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE configuration>
<configuration>
  <import class="ch.qos.logback.classic.encoder.JsonEncoder"/>
  <import class="ch.qos.logback.core.rolling.FixedWindowRollingPolicy"/>
  <import class="ch.qos.logback.core.rolling.RollingFileAppender"/>
  <import class="ch.qos.logback.core.rolling.SizeBasedTriggeringPolicy"/>

  <appender name="RollingFile" class="RollingFileAppender">
    <file>app.log</file>
    <rollingPolicy class="FixedWindowRollingPolicy">
      <fileNamePattern>app-%i.log</fileNamePattern>
      <minIndex>1</minIndex>
      <maxIndex>10</maxIndex>
    </rollingPolicy>
    <triggeringPolicy class="SizeBasedTriggeringPolicy">
      <maxFileSize>5MB</maxFileSize>
    </triggeringPolicy>
    <encoder class="JsonEncoder"/>
  </appender>

  <root level="DEBUG">
    <appender-ref ref="SOCKET"/>
  </root>
</configuration>
```

Usage of the logger at code level:

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
...
// Most common way to declare a logger
Logger logger = LoggerFactory.getLogger(MyClass.class);
// GOOD!
//
// Use parameterized logging to add user data to a message
// The pattern should be a compile-time constant
logger.warn("Login failed for user {}", username);
```

```
// BAD!
//
// Don't mix string concatenation and parameters
// If `username` contains `{}`, the exception will leak into the message
logger.warn("Failure for user " + username + " and role {}. ", role, ex);
...
```

References

- [Log4j Core Configuration File](#)
- [Log4j JSON Template Layout](#)
- [Log4j Appenders](#)
- [Logback Configuration File](#)
- [Logback JsonEncoder](#)
- [Logback Appenders](#)

Cryptography

General cryptography guidance

- **Never, ever write your own cryptographic functions.**
- Wherever possible, try and avoid writing any cryptographic code at all. Instead try and either use pre-existing secret management solutions or the secret management solution provided by your cloud provider. For more information, see the [OWASP Secrets Management Cheat Sheet](#).
- If you cannot use a pre-existing secret management solution, try and use a trusted and well known implementation library rather than using the libraries built into JCA/JCE as it is far too easy to make cryptographic errors with them.
- Make sure your application or protocol can easily support a future change of cryptographic algorithms.
- Use your package manager wherever possible to keep all of your packages up to date. Watch the updates on your development setup, and plan updates to your applications accordingly.
- We will show examples below based on Google Tink, which is a library created by cryptography experts for using cryptography safely (in the sense of minimizing common mistakes made when using standard cryptography libraries).

Encryption for storage

Follow the algorithm guidance in the [OWASP Cryptographic Storage Cheat Sheet](#).

Symmetric example using Google Tink

Google Tink has documentation on performing common tasks.

For example, this page (from Google's website) shows [how to perform simple symmetric encryption](#).

The following code snippet shows an encapsulated use of this functionality:

[Click here to view the "Tink symmetric encryption" code snippet.](#)

```
import static java.nio.charset.StandardCharsets.UTF_8;

import com.google.crypto.tink.Aead;
import com.google.crypto.tink.InsecureSecretKeyAccess;
import com.google.crypto.tink.KeysetHandle;
import com.google.crypto.tink.TinkJsonProtoKeysetFormat;
import com.google.crypto.tink.aead.AeadConfig;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.Base64;

// AesGcmSimpleTest
public class App {

    // Based on example from:
    // https://github.com/tink-crypto/tink-java/tree/main/examples/aead

    public static void main(String[] args) throws Exception {

        // Key securely generated using:
        // tinkey create-keyset --key-template AES128_GCM --out-format JSON --
        // out aead_test_keyset.json

        // Register all AEAD key types with the Tink runtime.
        AeadConfig.register();

        // Read the keyset into a KeysetHandle.
        KeysetHandle handle =
            TinkJsonProtoKeysetFormat.parseKeyset(
                new String(Files.readAllBytes(
                    Paths.get("/home/fredbloggs/aead_test_keyset.json")), UTF_8),
                InsecureSecretKeyAccess.get());

        String message = "This message to be encrypted";
        System.out.println(message);

        // Add some relevant context about the encrypted data that should be
        // verified
        // on decryption
        String metadata = "Sender: fredbloggs@example.com";

        // Encrypt the message
        byte[] cipherText = AesGcmSimple.encrypt(message, metadata, handle);
        System.out.println(Base64.getEncoder().encodeToString(cipherText));

        // Decrypt the message
        String message2 = AesGcmSimple.decrypt(cipherText, metadata, handle);
        System.out.println(message2);
    }
}

class AesGcmSimple {

    public static byte[] encrypt(String plaintext, String metadata,
```

```
KeysetHandle handle) throws Exception {  
    // Get the primitive.  
    Aead aead = handle.getPrimitive(Aead.class);  
    return aead.encrypt(plaintext.getBytes(UTF_8),  
        metadata.getBytes(UTF_8));  
}  
  
    public static String decrypt(byte[] ciphertext, String metadata,  
        KeysetHandle handle) throws Exception {  
        // Get the primitive.  
        Aead aead = handle.getPrimitive(Aead.class);  
        return new String(aead.decrypt(ciphertext,  
            metadata.getBytes(UTF_8)), UTF_8);  
    }  
}
```

Symmetric example using built-in JCA/JCE classes

If you absolutely cannot use a separate library, it is still possible to use the built JCA/JCE classes but it is strongly recommended to have a cryptography expert review the full design and code, as even the most trivial error can severely weaken your encryption.

The following code snippet shows an example of using AES-GCM to perform encryption/decryption of data.

A few constraints/pitfalls with this code:

- It does not take into account key rotation or management which is a whole topic in itself.
- It is important to use a different nonce for every encryption operation, especially if the same key is used. For more information, see [this answer on Cryptography Stack Exchange](#).
- The key will need to be stored securely.

[Click here to view the "JCA/JCE symmetric encryption" code snippet.](#)

```
import java.nio.charset.StandardCharsets;
import java.security.SecureRandom;
import javax.crypto.spec.*;
import javax.crypto.*;
import java.util.Base64;

// AesGcmSimpleTest
class Main {

    public static void main(String[] args) throws Exception {
        // Key of 32 bytes / 256 bits for AES
        KeyGenerator keyGen = KeyGenerator.getInstance(AesGcmSimple.ALGORITHM);
        keyGen.init(AesGcmSimple.KEY_SIZE, new SecureRandom());
        SecretKey secretKey = keyGen.generateKey();

        // Nonce of 12 bytes / 96 bits and this size should always be used.
        // It is critical for AES-GCM that a unique nonce is used for every
        cryptographic operation.
        byte[] nonce = new byte[AesGcmSimple.IV_LENGTH];
        SecureRandom random = new SecureRandom();
        random.nextBytes(nonce);

        var message = "This message to be encrypted";
        System.out.println(message);

        // Encrypt the message
        byte[] cipherText = AesGcmSimple.encrypt(message, nonce, secretKey);
        System.out.println(Base64.getEncoder().encodeToString(cipherText));

        // Decrypt the message
        var message2 = AesGcmSimple.decrypt(cipherText, nonce, secretKey);
        System.out.println(message2);
    }
}

class AesGcmSimple {

    public static final String ALGORITHM = "AES";
    public static final String CIPHER_ALGORITHM = "AES/GCM/NoPadding";
    public static final int KEY_SIZE = 256;
    public static final int TAG_LENGTH = 128;
    public static final int IV_LENGTH = 12;

    public static byte[] encrypt(String plaintext, byte[] nonce, SecretKey
secretKey) throws Exception {
        return cryptoOperation(plaintext.getBytes(StandardCharsets.UTF_8),
nonce, secretKey, Cipher.ENCRYPT_MODE);
    }

    public static String decrypt(byte[] ciphertext, byte[] nonce, SecretKey
secretKey) throws Exception {
        return new String(cryptoOperation(ciphertext, nonce, secretKey,
Cipher.DECRYPT_MODE), StandardCharsets.UTF_8);
    }

    private static byte[] cryptoOperation(byte[] text, byte[] nonce, SecretKey
```

```
secretKey, int mode) throws Exception {  
    Cipher cipher = Cipher.getInstance(CIPHER_ALGORITHM);  
    GCMParameterSpec gcmParameterSpec = new GCMParameterSpec(TAG_LENGTH,  
    nonce);  
    cipher.init(mode, secretKey, gcmParameterSpec);  
    return cipher.doFinal(text);  
}  
  
}
```

Encryption for transmission

Again, follow the algorithm guidance in the [OWASP Cryptographic Storage Cheat Sheet](#).

Asymmetric example using Google Tink

Google Tink has documentation on performing common tasks.

For example, this page (from Google's website) shows [how to perform a hybrid encryption process](#) where two parties want to share data based on their asymmetric key pair.

The following code snippet shows how this functionality can be used to share secrets between Alice and Bob:

[Click here to view the "Tink hybrid encryption" code snippet.](#)

```
import static java.nio.charset.StandardCharsets.UTF_8;

import com.google.crypto.tink.HybridDecrypt;
import com.google.crypto.tink.HybridEncrypt;
import com.google.crypto.tink.InsecureSecretKeyAccess;
import com.google.crypto.tink.KeysetHandle;
import com.google.crypto.tink.TinkJsonProtoKeysetFormat;
import com.google.crypto.tink.hybrid.HybridConfig;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.Base64;

// HybridReplaceTest
class App {
    public static void main(String[] args) throws Exception {
        /*

        Generated public/private keypairs for Bob and Alice using the
        following tinkey commands:

        ./tinkey create-keyset \
        --key-template DHKEM_X25519_HKDF_SHA256_HKDF_SHA256_AES_256_GCM \
        --out-format JSON --out alice_private_keyset.json

        ./tinkey create-keyset \
        --key-template DHKEM_X25519_HKDF_SHA256_HKDF_SHA256_AES_256_GCM \
        --out-format JSON --out bob_private_keyset.json

        ./tinkey create-public-keyset --in alice_private_keyset.json \
        --in-format JSON --out-format JSON --out alice_public_keyset.json

        ./tinkey create-public-keyset --in bob_private_keyset.json \
        --in-format JSON --out-format JSON --out bob_public_keyset.json
        */

        HybridConfig.register();

        // Generate ECC key pair for Alice
        var alice = new HybridSimple(
            getKeysetHandle("/home/alicesmith/private_keyset.json"),
            getKeysetHandle("/home/alicesmith/public_keyset.json")
        );

        KeysetHandle alicePublicKey = alice.getPublicKey();

        // Generate ECC key pair for Bob
        var bob = new HybridSimple(
            getKeysetHandle("/home/bobjones/private_keyset.json"),
            getKeysetHandle("/home/bobjones/public_keyset.json")
        );

        KeysetHandle bobPublicKey = bob.getPublicKey();

        // This keypair generation should be reperformed every so often in
```

```

order to
    // obtain a new shared secret to avoid a long lived shared secret.

    // Alice encrypts a message to send to Bob
    String plaintext = "Hello, Bob!";

    // Add some relevant context about the encrypted data that should be
    verified
    // on decryption
    String metadata = "Sender: alicsmith@example.com";

    System.out.println("Secret being sent from Alice to Bob: " +
    plaintext);
    var cipherText = alice.encrypt(bobPublicKey, plaintext, metadata);
    System.out.println("Ciphertext being sent from Alice to Bob: " +
    Base64.getEncoder().encodeToString(cipherText));

    // Bob decrypts the message
    var decrypted = bob.decrypt(cipherText, metadata);
    System.out.println("Secret received by Bob from Alice: " + decrypted);
    System.out.println();

    // Bob encrypts a message to send to Alice
    String plaintext2 = "Hello, Alice!";

    // Add some relevant context about the encrypted data that should be
    verified
    // on decryption
    String metadata2 = "Sender: bobjones@example.com";

    System.out.println("Secret being sent from Bob to Alice: " +
    plaintext2);
    var cipherText2 = bob.encrypt(alicePublicKey, plaintext2, metadata2);
    System.out.println("Ciphertext being sent from Bob to Alice: " +
    Base64.getEncoder().encodeToString(cipherText2));

    // Bob decrypts the message
    var decrypted2 = alice.decrypt(cipherText2, metadata2);
    System.out.println("Secret received by Alice from Bob: " + decrypted2);
}

private static KeysetHandle getKeysetHandle(String filename) throws
Exception
{
    return TinkJsonProtoKeysetFormat.parseKeyset(
        new String(Files.readAllBytes( Paths.get(filename)), UTF_8),
        InsecureSecretKeyAccess.get());
}

class HybridSimple {

    private KeysetHandle privateKey;
    private KeysetHandle publicKey;

    public HybridSimple(KeysetHandle privateKeyIn, KeysetHandle publicKeyIn)
    throws Exception {
        privateKey = privateKeyIn;
        publicKey = publicKeyIn;
    }
}

```

```
public KeysetHandle getPublicKey() {
    return publicKey;
}

public byte[] encrypt(KeysetHandle partnerPublicKey, String message, String
metadata) throws Exception {

    HybridEncrypt encryptor =
partnerPublicKey.getPrimitive(HybridEncrypt.class);

    // return the encrypted value
    return encryptor.encrypt(message.getBytes(UTF_8),
metadata.getBytes(UTF_8));
}

public String decrypt(byte[] ciphertext, String metadata) throws Exception
{

    HybridDecrypt decryptor = privateKey.getPrimitive(HybridDecrypt.class);

    // return the encrypted value
    return new String(decryptor.decrypt(ciphertext,
metadata.getBytes(UTF_8)), UTF_8);
}

}
```

Asymmetric example using built-in JCA/JCE classes

If you absolutely cannot use a separate library, it is still possible to use the built JCA/JCE classes but it is strongly recommended to have a cryptography expert review the full design and code, as even the most trivial error can severely weaken your encryption.

The following code snippet shows an example of using Elliptic Curve/Diffie Helman (ECDH) together with AES-GCM to perform encryption/decryption of data between two different sides without the need the transfer the symmetric key between the two sides. Instead, the sides exchange public keys and can then use ECDH to generate a shared secret which can be used for the symmetric encryption.

Note that this code sample relies on the `AesGcmSimple` class from the [previous section](#).

A few constraints/pitfalls with this code:

- It does not take into account key rotation or management which is a whole topic in itself.
- The code deliberately enforces a new nonce for every encryption operation but this must be managed as a separate data item alongside the ciphertext.
- The private keys will need to be stored securely.
- The code does not consider the validation of public keys before use.
- Overall, there is no verification of authenticity between the two sides.

[Click here to view the "JCA/JCE hybrid encryption" code snippet.](#)

```
import java.nio.charset.StandardCharsets;
import java.security.SecureRandom;
import javax.crypto.spec.*;
import javax.crypto.*;
import java.util.*;
import java.security.*;
import java.security.spec.*;
import java.util.Arrays;

// ECDHSimpleTest
class Main {
    public static void main(String[] args) throws Exception {

        // Generate ECC key pair for Alice
        var alice = new ECDHSimple();
        Key alicePublicKey = alice.getPublicKey();

        // Generate ECC key pair for Bob
        var bob = new ECDHSimple();
        Key bobPublicKey = bob.getPublicKey();

        // This keypair generation should be reperformed every so often in
        order to
        // obtain a new shared secret to avoid a long lived shared secret.

        // Alice encrypts a message to send to Bob
        String plaintext = "Hello"; //, Bob!";
        System.out.println("Secret being sent from Alice to Bob: " +
        plaintext);

        var retPair = alice.encrypt(bobPublicKey, plaintext);
        var nonce = retPair.getKey();
        var cipherText = retPair.getValue();

        System.out.println("Both cipherText and nonce being sent from Alice to
        Bob: " + Base64.getEncoder().encodeToString(cipherText) + " " +
        Base64.getEncoder().encodeToString(nonce));

        // Bob decrypts the message
        var decrypted = bob.decrypt(alicePublicKey, cipherText, nonce);
        System.out.println("Secret received by Bob from Alice: " + decrypted);
        System.out.println();

        // Bob encrypts a message to send to Alice
        String plaintext2 = "Hello"; //, Alice!";
        System.out.println("Secret being sent from Bob to Alice: " +
        plaintext2);

        var retPair2 = bob.encrypt(alicePublicKey, plaintext2);
        var nonce2 = retPair2.getKey();
        var cipherText2 = retPair2.getValue();
        System.out.println("Both cipherText2 and nonce2 being sent from Bob to
        Alice: " + Base64.getEncoder().encodeToString(cipherText2) + " " +
        Base64.getEncoder().encodeToString(nonce2));

        // Bob decrypts the message
```

```

        var decrypted2 = alice.decrypt(bobPublicKey, cipherText2, nonce2);
        System.out.println("Secret received by Alice from Bob: " + decrypted2);
    }
}

class ECDHSimple {
    private KeyPair keyPair;

    public class AesKeyNonce {
        public SecretKey Key;
        public byte[] Nonce;
    }

    public ECDHSimple() throws Exception {
        KeyPairGenerator keyPairGenerator = KeyPairGenerator.getInstance("EC");
        ECGenParameterSpec ecSpec = new ECGenParameterSpec("secp256r1"); //
        Using secp256r1 curve
        keyPairGenerator.initialize(ecSpec);
        keyPair = keyPairGenerator.generateKeyPair();
    }

    public Key getPublicKey() {
        return keyPair.getPublic();
    }

    public AbstractMap.SimpleEntry<byte[], byte[]> encrypt(Key
partnerPublicKey, String message) throws Exception {

        // Generate the AES Key and Nonce
        AesKeyNonce aesParams = generateAESParams(partnerPublicKey);

        // return the encrypted value
        return new AbstractMap.SimpleEntry<>(
            aesParams.Nonce,
            AesGcmSimple.encrypt(message, aesParams.Nonce, aesParams.Key)
        );
    }

    public String decrypt(Key partnerPublicKey, byte[] ciphertext, byte[]
nonce) throws Exception {

        // Generate the AES Key and Nonce
        AesKeyNonce aesParams = generateAESParams(partnerPublicKey, nonce);

        // return the decrypted value
        return AesGcmSimple.decrypt(ciphertext, aesParams.Nonce,
aesParams.Key);
    }

    private AesKeyNonce generateAESParams(Key partnerPublicKey, byte[] nonce)
throws Exception {

        // Derive the secret based on this side's private key and the other
side's public key
        KeyAgreement keyAgreement = KeyAgreement.getInstance("ECDH");
        keyAgreement.init(keyPair.getPrivate());
        keyAgreement.doPhase(partnerPublicKey, true);
        byte[] secret = keyAgreement.generateSecret();

        AesKeyNonce aesKeyNonce = new AesKeyNonce();

        // Copy first 32 bytes as the key
        byte[] key = Arrays.copyOfRange(secret, 0, (AesGcmSimple.KEY_SIZE /

```

```
8));  
    aesKeyNonce.Key = new SecretKeySpec(key, 0, key.length, "AES");  
  
    // Passed in nonce will be used.  
    aesKeyNonce.Nonce = nonce;  
    return aesKeyNonce;  
  
}  
  
private AesKeyNonce generateAesParams(Key partnerPublicKey) throws  
Exception {  
  
    // Nonce of 12 bytes / 96 bits and this size should always be used.  
    // It is critical for AES-GCM that a unique nonce is used for every  
    cryptographic operation.  
    // Therefore this is not generated from the shared secret  
    byte[] nonce = new byte[AesGcmSimple.IV_LENGTH];  
    SecureRandom random = new SecureRandom();  
    random.nextBytes(nonce);  
    return generateAesParams(partnerPublicKey, nonce);  
  
}  
}
```